SEASUG Paper 125-2026

Creating Wide Complex Output Tables While Keeping One's Sanity

Nat Wooding, J Sargeant Reynolds Community College

ABSTRACT

The programmer is sometimes asked to create wide tables whose creation is more involved than doing a few simple joins and whose output can contain more than one record type, whose variables must be in a specific order, and whose variable names may not be easily handled in within a SAS ® program. Several methods are shown which aid in dealing with such situations. Examples will use one of the files required by the National Student Clearinghouse (NSC) Postsecondary Data Partnership (PDP) and they will primarily use data step code.

INTRODUCTION

In recent years I have been asked to prepare data tables for the National Postsecondary Student Aid Study (NPSAS) as well as those of the National Student Clearinghouse Postsecondary Data Partnership (PDP).

NSC uses the PDP files to populate dashboards that can display student body statistics and colleges use these dashboards to study their student body and work with such advisory organizations as Achieving the Dream (https://achievingthedream.org/). The NSC requires two different output files in comma separated value format and each of these requires one header and two trailer records in addition to the data lines. The output files are wide enough that they can't be viewed without scrolling to the right so seeing the entire line is not easy or impossible. The PDP has two file types, the Cohort file which identifies students who first enrolled in a given semester and contains various demographic and other information; the Course file holds enrollment records during the semesters of each student's enrollment.

This paper shows several techniques that I have found to be helpful in dealing with such files. I will show examples mainly from the preparation of the PDP's Cohort file.

One technique to be demonstrated is the use of the original database variables names or simple names as aliases throughout the program until the very end. When we write the output file, we will use a macro variable to substitute the required names for those used in our program.

Reynolds uses Peoplesoft's Student Information System student records system (SIS) but the techniques should be useable with any data source.

PDP NUANCES

NSC requires that variables have specific names and that they be in a specific order. Furthermore, each of the two files must have two header records as well as a trailer record. Hence, writing to the output files is not a matter of simply writing out a SAS data set.

SIS NUANCES

A couple SIS variable names could use explanation. The student ID variable is called EMPLID – remember that the parent system, Peoplesoft, is a human resources system. Semesters are referred to as STRM.

SAS PROGRAMS TO BE USED

I have found it helpful to have separate programs to handle the Cohort and Course file creations. Since each of these must be told which semesters to store and how to name the output files, I use a short SAS program to store macro variables that may be used by each of these programs and then use a %INCLUDE statement to bring these into my programs. Also, I store a SAS data set for each Cohort data submittal to make this available later to the Courses program.

THE COHORT FILE STRUCTURE

The cohort file is written in UTF-8 and comma separated. The file has four record types and each line starts with the appropriate record type indicator. The first line basically identifies the college and the particular submittal. Then there is a line that gives column header labels and these are often sets of words with embedded blanks. Figure 1 shows the first line (DCE02) and a portion of the second line (CH1). Following the header records are the actual data lines and finally there is a line (T1) that marks the end of the file and gives the total record count. Figure 1 shows parts of a sample Cohort file.

DCE02,10087261,003759,00,,,,,20260819,,ReynoldsCC_COURSE_Load_July_2026_Submit,,,,

CH1,Cohort,Cohort Term,Academic Year,Term,Institution ID Type,Institution ID,SSN,

…

T1,8

Figure 1. Snip of a Cohort File

In Figure 1, the two header lines appear. The first item in each line identifies the particular line type with the first line identifying the specific filetype that we are submitting and the second identifies the variables. The ellipsis represents the contents of the SAS data set containing our student information. The T1 line identifies the end of the file and gives the total record count.

THE START OF THE COHORT PROGRAM

This program has some initial data steps which bring in stored macro variables and then create more macro variables for use in later steps.

These are followed by several lines that create a macro variable which will be used for the header record:

```
Data  _Null_ ;    * Set up some macro variables for the job;
    S1     = 'DCE01,10087261,003759,00, , , ,';
    Date   = Compress ( Put ( today() , YYMMDD10.) , '-' );
    Filler = '@';
    FileID = Compress ( "i_10087261PDP_ReynoldsCommunityCollege_Cohort_&Term._Submitted_" ||
Date || '.txt' );*Unique to JSR;
    String0 = "ReynoldsCC_COHORT_&Term._Submit";
    String = catx ( ',' , S1 , Date , Filler , String0, Filler , Filler , Filler , Filler );
    String = Translate ( String  , ' ' , '@');
    String = Compress ( String ); * NSC was getting upset over some blank fields;
    call symput ( 'F_header' , String );
    call symput ( 'FileID'   , FileID );
* The PDP file is specific about fields and blanks and the somewhat cumbersome approach taken
ensured that the string being generated would meet the requirements;
run;
```

With this done, I create a wide macro variable that holds the variable names used by the PDP:

%let Header = CH1,Cohort,Cohort Term,Cohort Term Begin Date … ;

I simply copied the list from the PDP User's Guide (https://help.studentclearinghouse.org/pdp/wp-content/uploads/PDPDataSubmissionGuide.pdf) and added commas between the variables. Doing this eliminated the risk of mistyping or omitting variables.

We will use this later when we write the header records.

SIS tables containing enrollment information use STRM (the semester) as a primary key and we will need to convert the individual term values into such variables as academic year. Fortunately, the college maintains an SIS table that shows the academic year in which each STRM belongs and I use these data to create formats that we use later when we need to identify the academic year and also when it starts and stops. Note that since the SIS table is updated regularly, I do not have to maintain a crosswalk myself. Here is one example of the Proc Format steps:

```
Data Formats2;     * identify beginning date for an STRM;
    set sis.ps_term_tbl keep = STRM TERM_BEGIN_DT ) ;
    length label $25.;
    start= STRM  ;
    retain fmtname 'Bgn'  type 'C';
    label = Put ( datepart (TERM_BEGIN_DT) , yymmddn. ) ;
run;

Proc Format cntlin = Formats2  ;
run;
```

THE SAS PROGRAM DATA VECTOR (PDV)

When SAS encounters a new Data step, it populates a temporary data set in memory which holds the metadata for each variable in the set. This includes, of course, the variable's name, its type (numeric or character), flags to indicate whether to retain values and whether to keep or drop the variable when each record is output. A very important feature is that the individual variable is added sequentially and that the order of the variable cannot be changed during the execution of the data step. Variables are added as sets are read (or raw data input using an input statement) and as variables are created within the code. Hence, if we need to have our variables in a specific order and the SAS Supervisor would encounter the variables in a different order, we must first identify these variables using a LENGTH statement and place this statement before a SET, MERGE or INPUT statement. These aspects of the PDV creation will be important as we start to build our output data and keep the variables in their proper order.

CREATING INDIVIDUAL STUDENT RECORDS

Up to now, we have been setting up macro variables for later use and, also, preparing user-defined formats for use while processing our data. Much of this could have been mixed in the subsequent data steps but my preference is to place such code at the start of the program.

In the following notes, I will describe several techniques that I use to make it easier to ensure that we have the needed variables, that we can readily identify them, and that they are in the proper order.

First, let's see what we are trying to produce. Here is a snippet of a few student records using fake data. These come from our first data set and it defines the first seven variables which we will output. Note that two of the variables will not be populated.

	A_F_type	B_Cohort	C_Cohort_trm	D_Cohort_trm_Bgn	E_Cohort_trm_End	F_SSN	G_ITIN	EMPLID
1	D1	2023-24	Spring	20240116	20240506			9918884
2	D1	2023-24	Spring	20240116	20240506			9922269
3	D1	2019-20	Fall	20190826	20191214			9945494
4	D1	2023-24	Summer	20240520	20240731			9982189
5	D1	2022-23	Fall	20220822	20221212			9983639

Figure 2. An Intermediate SAS Data Set with Some of the Variables

The first variable identifies this as a student cohort record.

Note that most variables shown here have a letter prefix (two letters if we have more than 26 output columns) followed by an underscore, the exception being the last, "EMPLID". The letters will correspond to the column labels that we would see if we were to open our output CSV in Excel. When we are creating our program, this will let us immediately see whether our variables have been introduced in the proper order and whether there are any variables that we should have dropped which in this case is EMPLID, the student ID. I am keeping this spelling of the variable since later steps will gather other variables and we will need to merge the various intermediate files using EMPLID. In a later data step, we will create a copy using the prefixed name and located in the proper column.

This use of prefixes is probably the most important point of the presentation.

Before I found this approach of using the column-identifying prefixes, I found myself getting mental whiplash trying to verify that I had the proper variables and they were in the proper order.

Previously, I mentioned that the order in which variables are first mentioned determines their location in the data set. Here is an example of a data step where we use a length statement to insert a variable in the PDV before data are actually added:

```
Data Cohort0; *Start the initial SAS data set, specify the order of the variables, create variables;
  Length A_F_type $3 B_Cohort $7 C_Cohort_trm $6  D_Cohort_trm_Bgn  E_Cohort_trm_End $8 _SSN
G_ITIN  $ 9 ;
  set Emplid1;
    B_Cohort                = AY;
    C_Cohort_Trm            = Put ( strm     , $trm.      );
    D_Cohort_trm_Bgn        = put ( strm     , bgn.       );
    E_Cohort_trm_End        = put ( strm     , end.       );
   Retain A_F_Type 'D1'    F_SSN G_ITIN '          ' ;     Run;
```

Here we create the variables shown in the data snippet. We read the set EMPLID1 but want to force the order of out output data variables before we read in this data set so we use a length statement before the Set statement. The first variable which identifies each observation as being a data line is identified first but not mentioned again until the last line of the data step. The first mention determines its order in the PDV and the Retain statement sets a flag telling SAS to not set the value to missing at the start of each new observation but, rather, to always store the value “D1”. The retain statement’s location doesn’t matter since the PDV is defined prior to actually processing input data. The three variables that begin with C, D, and E, are all populated using STRM, formats and PUT statements which translate STRM into various other values. Reynolds do not use the F_SSN and G_ITN variables to identify our students so they are identified in their proper order and set to missing.

The subsequent data steps use similar techniques to gather the remaining variables which we report to the PDP and eventually, these are combined into one long SAS data set which we will write to the CSV.

CREATING THE OUTPUT FILE

Here instead of PROC EXPORT, we will use the old-school PUT statement which gives us quite a bit of control over the process.

First, of course, we need to tell SAS where to store the output so we use a FILENAME statement:

Filename out "&FilePath\&Folder_term\&Fileid" encoding="UTF-8";

We use three previously created macro variables to specify three of the nodes in the path and we specify the encoding that NSC requires.

Recall that we are using aliases in place of the actual variable names that NSC demands so we will need to substitute the NSC names. Since we will use a PUT statement to write out our data lines and the PUT statement does not identify the variable names, we will have the PUT write out a string containing the needed names. To do this, we need to create the string holding the desired names and we do this dynamically:

Proc contents data = Cohort_Table0 noprint out = Varlist (keep = name);

run; *Start with a list of all of the variables that we have collected;

*We could have used dictionary tables here;

Data Varlist; *Create a macro variable with the variables that we want to output. This code skips those that lack the letter and underscore prefixes;

set varlist end = eof

Length string $ **1000**;

Retain String;

if index (substr (name , **2** , **3**) , '_') then keep = **1** ;*Identify the variables that have an underscore in the second or third position;

If keep then String = Compbl (String || name);

if eof ;

Call symput ('OutVars' , String);* create a macro variable after we have read all of our variable names and selected those that we want ;

run;

Note that SAS did not simply delete unwanted variables as I encountered them since if the last variable was not wanted, our EOF flag would not have been seen.

```
Data WriteBody  ;                          *  Write out the output file;
   set Cohort_Table0 ( Keep = &OutVars   ) End = EOF  ;
   File out dlm = ',' ;
   If _n_ = 1 then Put "&F_header" / "&Header";* First, write the header lines;
   Put ( _all_ )  (+0)   ;                 * Now, the body of the table;
   if eof;                                 * find the last record;
   Put 'T1,' "&Records" ',';               * And, finally, the last line of the output file which is the record count of
the file;
run;
```

The macro variable &Records was created in a short prior data step that is not shown.

THE FINAL STEP

At this point, we have created our Cohort file and it is ready to be submitted. Next, we run must the Courses program which collects information on the classes that our cohort students have taken in the most recent semester. It needs to know which students to include as well as some other information from the Cohort file so we store the Cohort records as a SAS data set. To read these, the Cohort program stores the student records from each data submittal in a folder and the Courses program reads and combines these SAS data sets in order to know which student course records to retrieve. Since Courses uses some of the variables from Cohort but not in the same order as Cohort, LENGTH statements are very useful to determine their proper location in the PDV.

CONCLUSION

Some simple data step programming can be used to create somewhat complex data output. Variable name aliases with prefixes let us select the variables that we want and also verify that they are in the proper location.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Nat Wooding
J Sargeant Reynolds Community College
Nwooding@reynolds.edu